Department of Information Technology
College of Engineering, Guindy
Anna University, Chennai-25

LONG-TERM MEMORY ENGINE FOR AI AGENTS

Second Project Review

GUIDE:

DR. BAMA SRINIVASAN
DEPARTMENT OF INFORMATION TECHNOLOGY
CEG, ANNA UNIVERSITY

TEAM MEMBERS:

M. KRITHIKRAJ, 2023115004
ALSTON S, 2023115006
ARYAN I, 2023115021
HARI SRINIVASAN, 2023115022

PROBLEM STATEMENT

- An AI agent forgets everything when a session ends. Instructions, decisions and corrections from earlier sessions are lost unless the user repeats them.
- Pasting the whole history into the prompt does not solve this. Models use information in the middle of a long context poorly [1], and every answer gets slower and costlier as the history grows.
- When a detail changes, the agent cannot tell the new value from the old one, so it may act on outdated information.
- We need a memory that keeps what the user said, notices when it changes, and returns only the relevant parts with the message each came from. LongMemEval [2] tests exactly this: recall across sessions, reasoning about time, handling updates and admitting when it does not know.

Example

Session 1: "The release freeze is Thursday at 17:00."
Session 2: "The freeze moved to Friday at 16:30."
Session 3: "When is the release freeze?" The agent should say Friday at 16:30, and know it used to be Thursday.

LITERATURE SURVEY (1/2)

Work	How it works	Reported results	Limitation
MemGPT, Packer et al., 2023 [3]	Splits the prompt into a main context and outside storage. The model moves data in and out with function calls.	92.5% on deep memory retrieval with GPT-4, against 32.1% for GPT-4 reading a summary.	Memory changes only when the model makes the right call. No latency or cost figures.
Zep / Graphiti, Rasmussen et al., 2025 [4]	Temporal knowledge graph of episodes, entities and edges. Each edge has a valid and an invalid time, and a contradiction closes the old edge.	71.2% on LongMemEval with gpt-4o, against 60.2% for the full 115k-token history.	One run with one configuration. The graph database is the main store.
Mem0, Chhikara et al., 2025 [5]	An LLM extracts candidate memories, compares each with similar stored ones and picks add, update, delete or no change.	LoCoMo judge score 66.9, against 72.9 for the full history. p95 latency 1.4 s against 17 s.	One benchmark and an unnamed judge model. The open-source release has dropped update, delete and graph memory.

LITERATURE SURVEY (2/2)

Work	How it works	Reported results	Limitation
Supermemory, Daga et al., 2026 [6]	Atomic memories linked to source chunks, with "updates", "extends" and "derives" links and two timestamps.	LongMemEval-S scores by category, from the vendor's own run.	Closed source. The report is not peer reviewed and some of its metrics are not defined.
Hindsight, Latimer et al., 2025 [7]	Retain, recall and reflect operations. Recall fuses four retrieval strategies by rank and packs the result into a token budget.	LongMemEval and LoCoMo scores from the authors' own runs.	Not peer reviewed. No ablation or variance, and the token budget is not stated.
Full history in the prompt, Liu et al., 2024 [1]	No separate memory. The whole conversation goes into the model's context.	Accuracy drops when the needed passage sits in the middle of a long input.	Slower and costlier as history grows. Nothing marks a value as outdated.

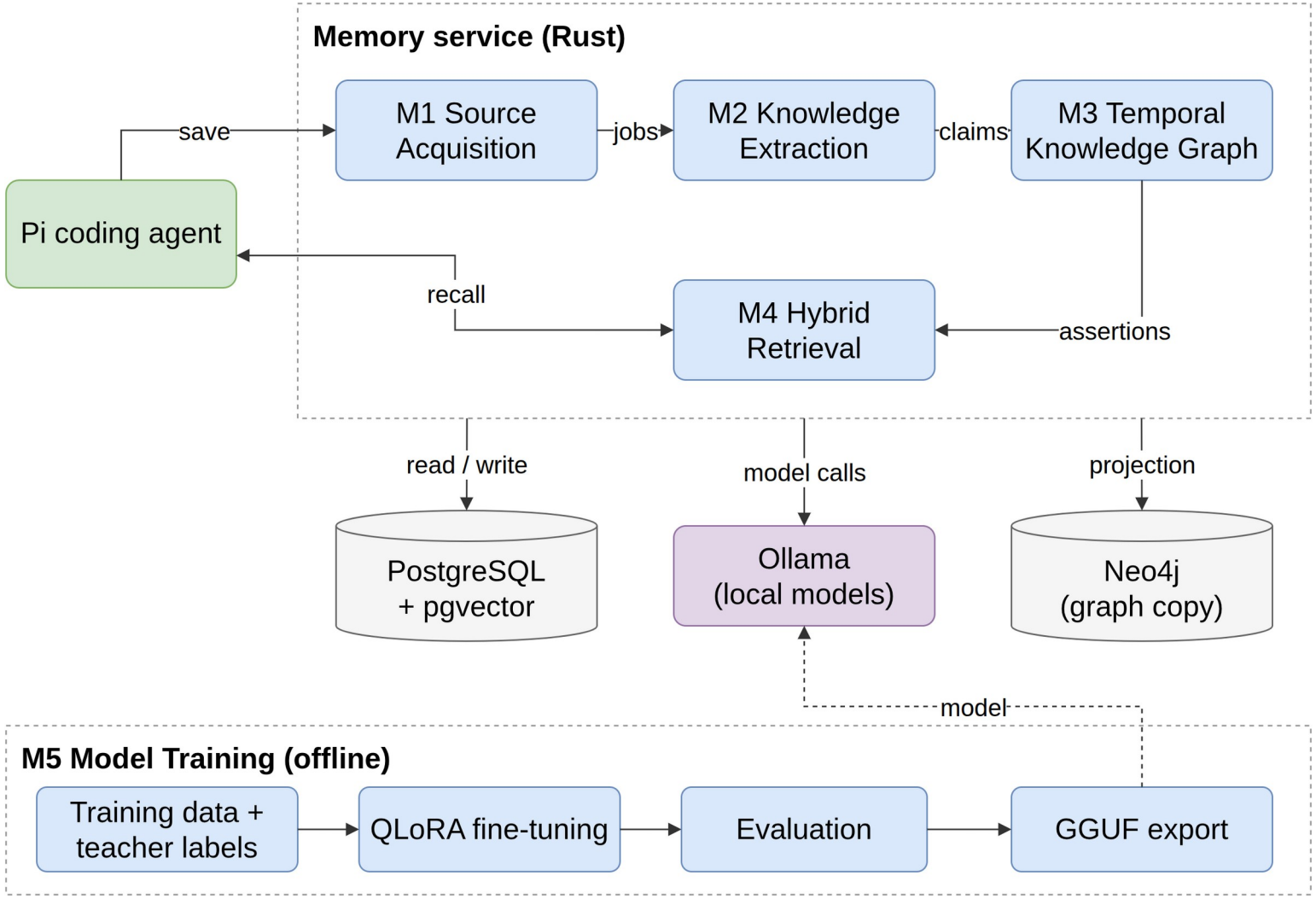
CHALLENGES / ISSUES IDENTIFIED: GAPS IN THE LITERATURE

Gap in existing work	Seen in	How we address it
The model decides how memory changes, so a missed or wrong call silently loses or overwrites a memory.	MemGPT [3], Mem0 [5]	Update rules are plain service code: one value or many per slot, plus a reviewed correction flag.
Changed values are overwritten or deleted, so there is no record of what used to be true.	Mem0 [5], MemGPT [3]; Zep [4] keeps it only in its graph	Each change is a new version with a validity interval in PostgreSQL. The graph is a copy we can rebuild.
Stored memories are hard to check against what the user actually said.	MemGPT [3], Mem0 [5]; Supermemory [6] is closed	Every assertion cites an exact quote, and the quote is matched against the original message.
Extraction depends on large hosted models, which adds cost, latency and privacy concerns.	Zep [4], Mem0 [5]	A fine-tuned 4B model does extraction, consolidation and answering locally through Ollama.
Results are hard to compare: single runs, vendor-run benchmarks, different judges and unstated budgets.	Zep [4], Mem0 [5], Supermemory [6], Hindsight [7]	We publish our own test cases and run LongMemEval-S with fixed settings.

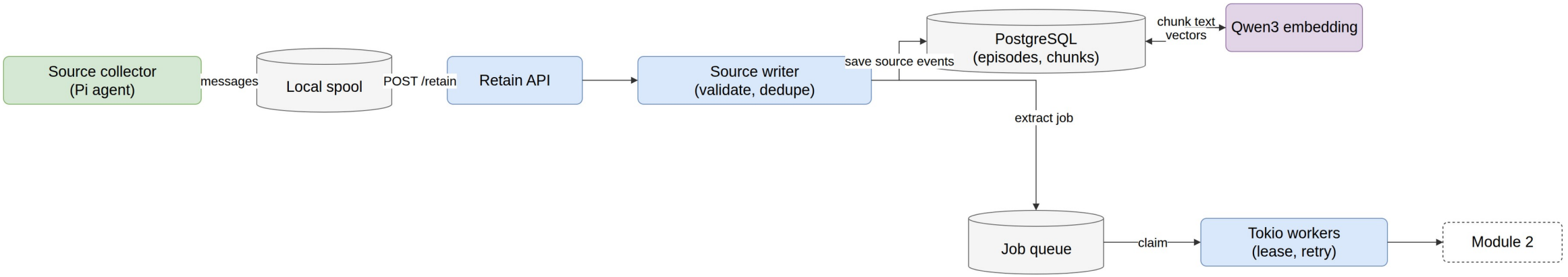
OBJECTIVES OF THE PROJECT

- Keep useful information for a general-purpose AI agent across sessions.
- Keep the latest version of an instruction without losing the earlier ones.
- Link every stored assertion to the message it came from, so it can be checked.
- Fine-tune a small local model to extract assertions, combine related ones and answer with citations.
- Return the relevant memories quickly, within a fixed context budget.
- Measure memory accuracy and retrieval quality with repeatable tests.

SYSTEM ARCHITECTURE

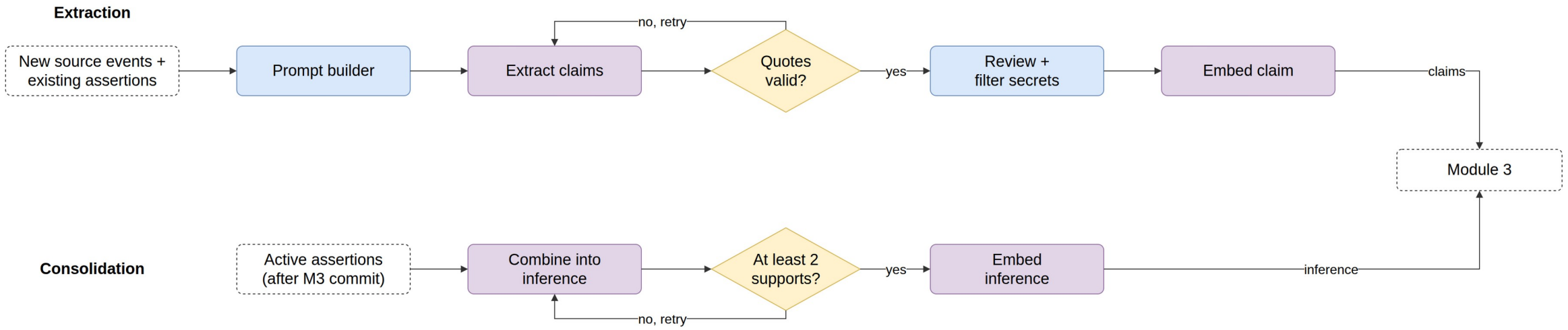


MODULE 1: SOURCE ACQUISITION AND PROCESSING



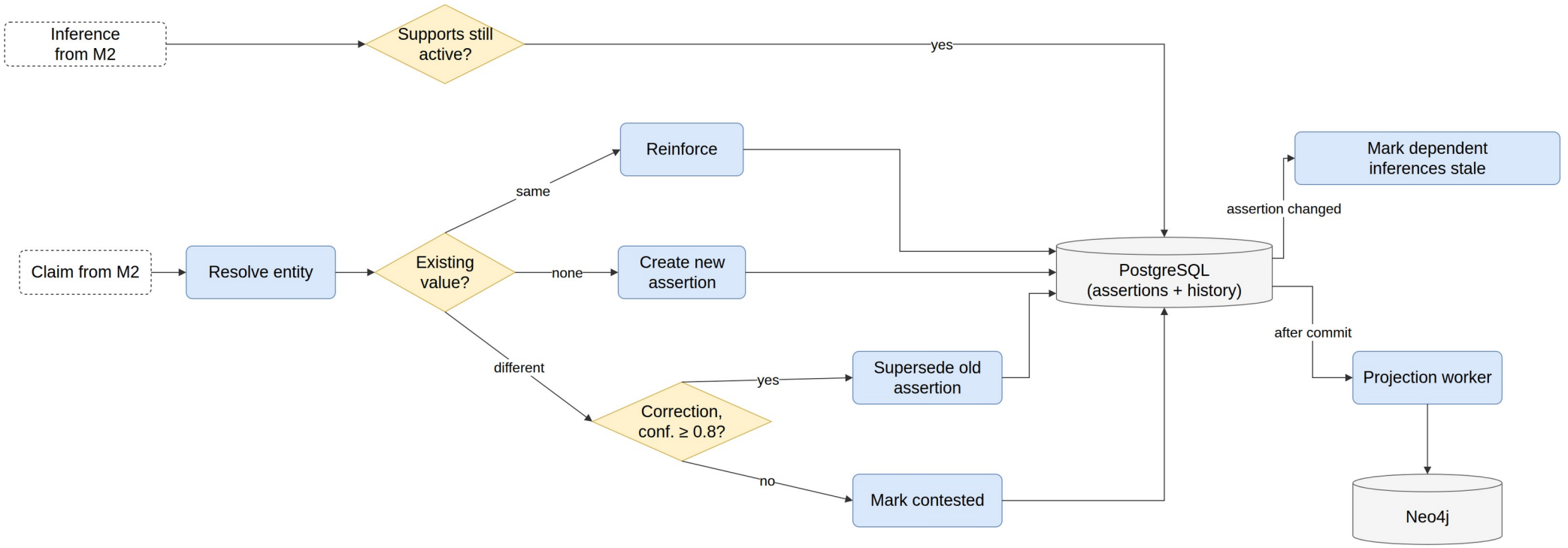
- Pi sends every message to a local spool first, so nothing is lost if the service is down.
- The retain API checks each batch, and a repeated request gets the existing IDs back instead of duplicates.
- Source events, chunks and an extraction job are saved in one PostgreSQL transaction.
- Chunks are embedded with Qwen3 Embedding, and Tokio workers lease jobs and retry up to five times.

MODULE 2: KNOWLEDGE EXTRACTION AND CONSOLIDATION



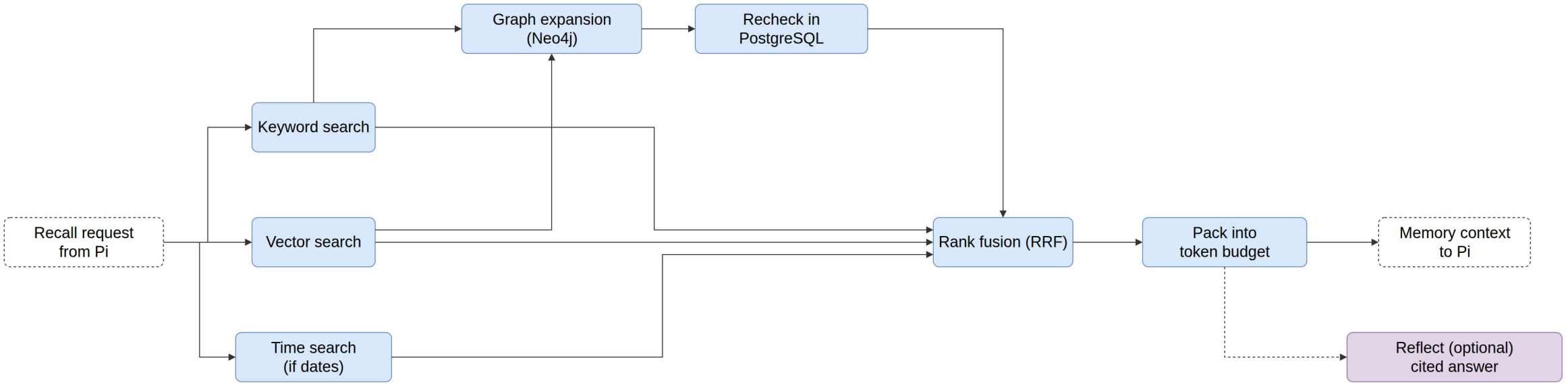
- Extraction: the fine-tuned model returns claims with exact quotes. A quote missing from the source is rejected and the model retries, up to two times.
- A review step confirms corrections and filters out secrets before claims go to Module 3.
- Consolidation: after Module 3 commits, two or more active assertions about a subject can become one inference.

MODULE 3: TEMPORAL KNOWLEDGE & GRAPH MANAGEMENT



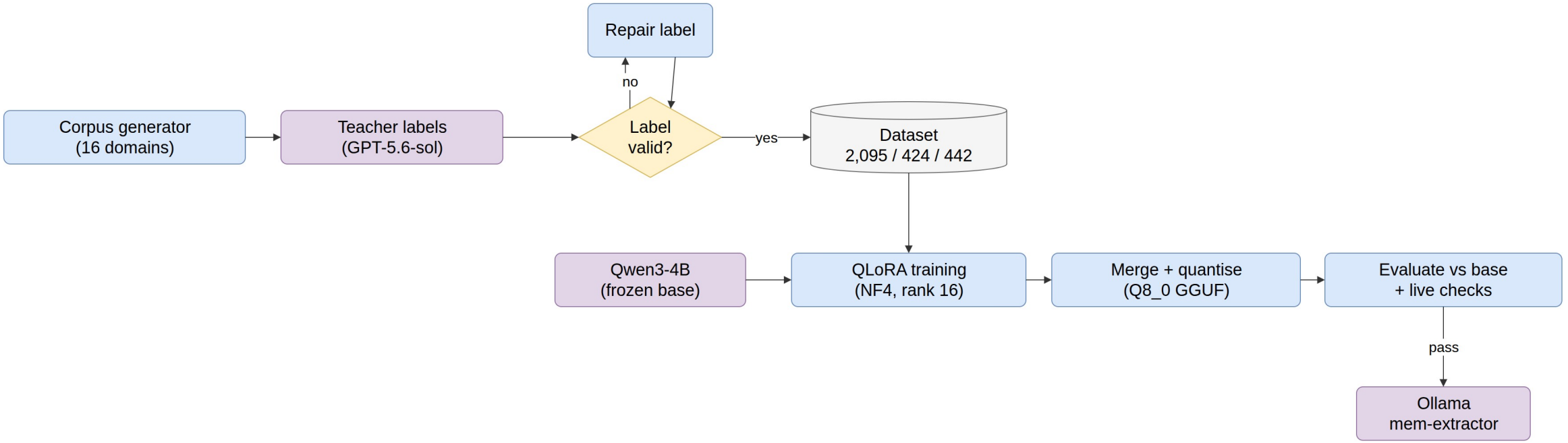
- The only module that writes assertions. A new value replaces the old one only if it is a reviewed correction with confidence of at least 0.8; otherwise both are marked contested.
- Old values keep their validity interval, and a projection worker copies every change to Neo4j.

MODULE 4: HYBRID RETRIEVAL AND GROUNDED REASONING



- Keyword, vector and, when the question has dates, time-based search run together. Top hits are expanded through Neo4j and checked again in PostgreSQL.
- Reciprocal Rank Fusion merges the lists, and the best results are packed into the token budget with their quotes.
- An optional reflect step writes an answer with citations.

MODULE 5: MODEL TRAINING AND EVALUATION



- Runs offline on synthetic episodes from 16 domains labelled by GPT-5.6-sol; live chats are never used for training.
- Qwen3-4B is fine-tuned with QLoRA, quantised to GGUF and served by Ollama only after it beats the base model.

ALGORITHMS OR METHODOLOGY

Algorithm	Purpose
1. Priority-based context selection	Picks the existing assertions that matter most and fit the model's input limit.
2. Entity resolution and memory versioning	Matches entities and handles repeats, corrections and conflicting values.
3. Memory consolidation and invalidation	Combines assertions into inferences and marks them stale when their support changes.
4. Hybrid retrieval and graph traversal	Finds memories by keyword, meaning, date and graph connection.
5. Reciprocal Rank Fusion and context packing	Merges the search rankings and fits the best memories into the context budget.
6. Source-grounded answer generation	Answers from retrieved memories and checks every citation.

ALGORITHMS OR METHODOLOGY

1. Priority-based context selection

Input: existing assertions, new messages, input budget

IF all existing assertions fit:

    RETURN all of them

Score each assertion by how often its subject and value appear in the new messages

Sort by score, newest first on ties

Binary-search the largest prefix that fits the budget

RETURN that prefix in its original order

2. Entity resolution and versioning

Input: a new claim and the stored memory

Match the entity by type and normalised name, or a unique alias; otherwise create it

IF the same assertion exists:

    attach the new quote

ELSE IF the slot has no conflicting value:

    store the claim as active

ELSE IF reviewed correction AND confidence ≥ 0.8:

    supersede the old value

ELSE:

    mark both values contested

ALGORITHMS OR METHODOLOGY

3. Consolidation and invalidation

Input: active assertions about the changed entities

Ask the model to combine related assertions into inferences

Keep an inference only if it has two or more supports

Check the support IDs and the subject

Recheck that every support is still active, then save with links

WHEN a support changes or is retracted:

    find dependent inferences recursively

    mark them stale

4. Hybrid retrieval and graph traversal

Input: a question and an optional date range

Resolve date phrases if planning is on

Run keyword search [8] and vector search [9]

IF there is a date window:

    add time-based candidates

Expand from the matched assertions and entities for two bounded hops

Drop results outside their validity or the distance cutoff

RETURN the candidate lists

ALGORITHMS OR METHODOLOGY

5. Rank fusion and context packing

Input: ranked candidate lists and a token budget

FOR each memory:

    score = Σ 1 / (60 + rank) over all lists [10]

Sort by score, then by ID

FOR each memory in order:

    build a block with the assertion and its quotes

    add it if it fits the budget

    stop at the requested count

RETURN the packed context

6. Source-grounded answering

Input: a question and the retrieved memories

IF nothing was retrieved:

    RETURN "insufficient evidence"

Ask the model to answer only from the retrieved memories

Allow citations only to retrieved IDs

Check the output format and every cited ID

IF it claims an answer without citations:

    reject it

RETURN the answer with its citations

TOOLS AND TECHNOLOGIES (1/2)

Tool	Role
Rust, Tokio, Axum and SQLx	The memory service: API, background workers and database access.
PostgreSQL 17 with pgvector [8], [9]	Source events, assertions, history and jobs; keyword and vector search.
Neo4j 5.26 [11]	Graph copy of assertions and entities, used to widen recall.
Ollama	Runs the memory model and the embedding model locally.
Qwen3-Embedding-0.6B [12]	Turns messages, assertions and questions into vectors.

TOOLS AND TECHNOLOGIES (2/2)

Tool	Role
Fine-tuned Qwen3-4B-Instruct-2507	Extracts assertions, combines them and answers with citations.
Python, PyTorch, Transformers, PEFT, bitsandbytes	Dataset preparation and QLoRA fine-tuning [14].
llama.cpp and GGUF	Converts and quantises the trained model for Ollama.
Pi coding agent [13]	Captures session messages and adds recalled memory to the agent's context.
React, TypeScript, Vite and Bootstrap	Dashboard, session controls and the memory graph view.
Docker Compose, Caddy and ttyd	Runs the services and gives browser access to the app and the Pi console.

DATASET (1/3)

Fine-tuning dataset (Qwen3-4B-Instruct-2507)

Aspect	Details
Scope	Started with coding conversations, then expanded to 16 domains such as family, food, travel, education, work and software.
Source	Synthetic conversations labelled by GPT-5.6-sol, checked by a second adversarial model and cleaned.
Size	2,961 task examples: 2,095 training, 424 validation and 442 test.
Tasks	Extract assertions, combine related assertions and answer questions with citations.

Training example

Assistant: "You could learn violin this winter."
User: "No, that is not my plan. I am learning watercolor painting."
Expected memory: The user is learning watercolor painting.

DATASET (2/3)

External evaluation dataset

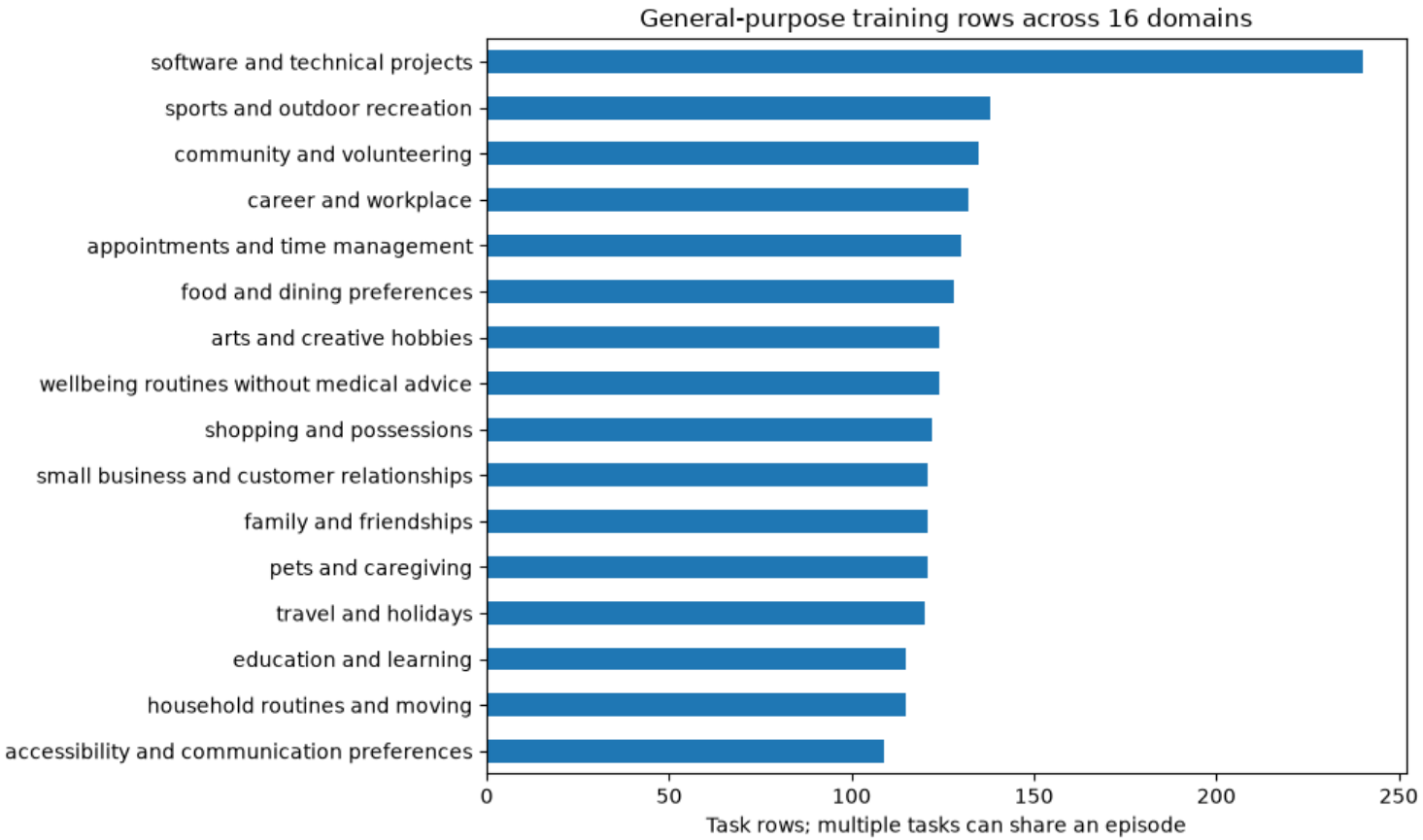
Aspect	Details
Source	LoCoMo [15]: long conversations spread over many sessions.
Size	16 conversation segments from 10 conversations, with 40 reference assertions.
Purpose	Check whether the model extracts useful assertions from conversations outside our training data.

Evaluation example (LoCoMo)

Caroline: "Did you make that plate?"
Melanie: "Yes, during yesterday's pottery class. I find pottery relaxing and creative."
Caroline: "I haven't tried pottery. I've been painting and recently finished a picture."
Melanie: "What inspired your painting?"
Caroline: "A visit to the beach last week inspired it."

Expected memory: Melanie made a plate in pottery class. Caroline painted a picture inspired by her beach visit.

DATASET (3/3)



Software and technical projects has 240 rows and the other domains have about 110 to 140 each. One episode can give several task rows.

OUTPUT SCREENSHOTS (1/2)

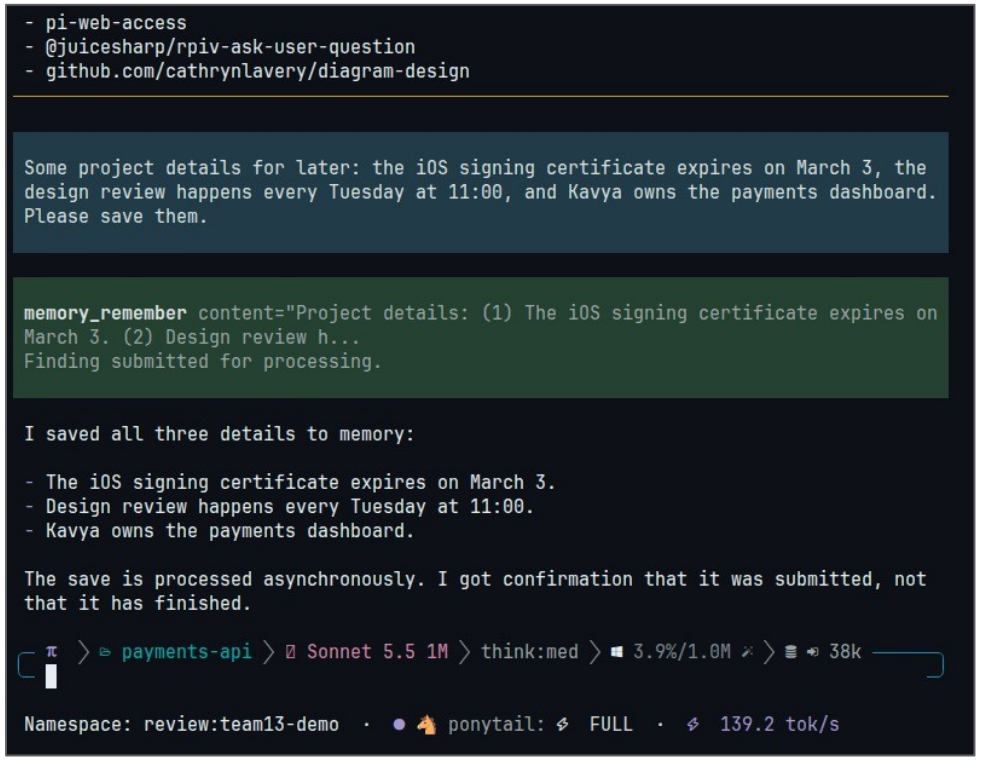


Figure 1. Recording: in the first session the user gives a certificate expiry date, a weekly review time and the dashboard owner, and Pi submits them to memory.

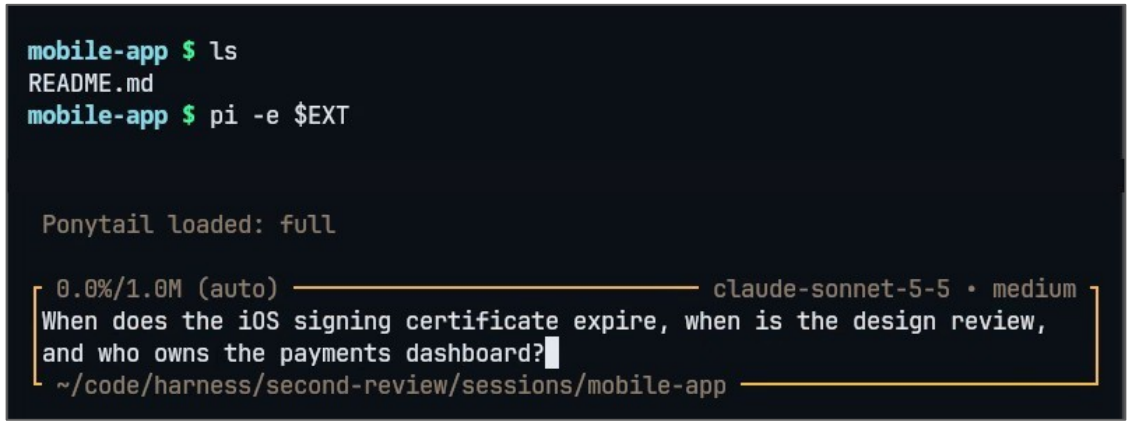


Figure 2. New session: a fresh session in another project folder asks for the same details without repeating them.

OUTPUT SCREENSHOTS (2/2)

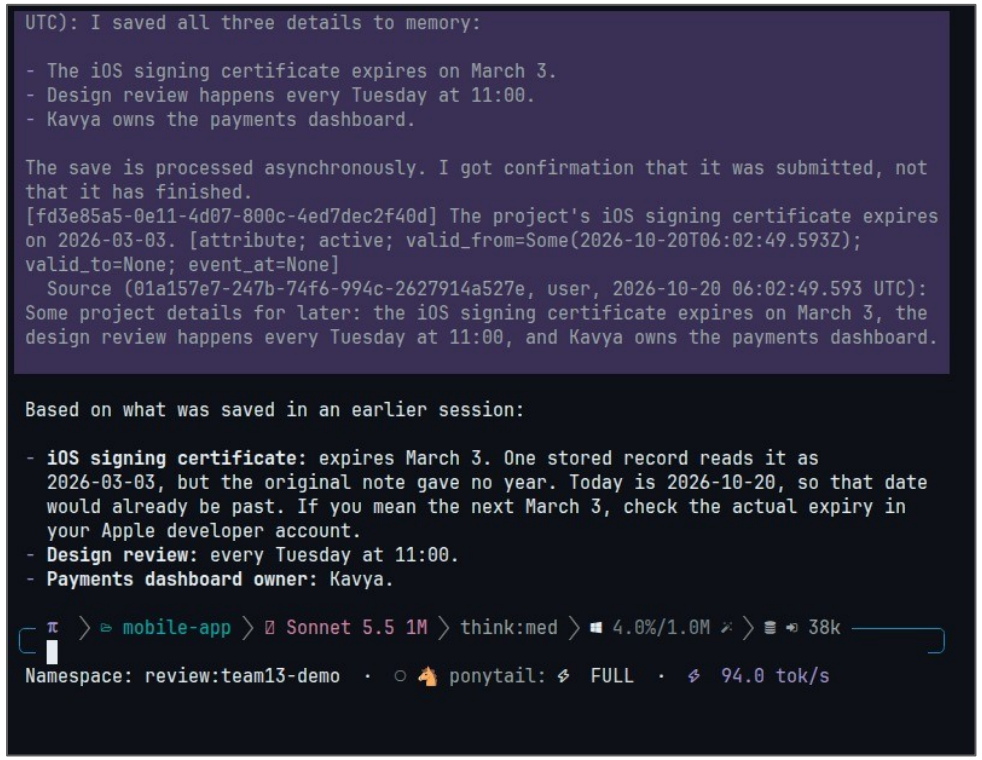


Figure 3. Recall: memory from the first session is added to the context, and the agent answers March 3, Tuesday at 11:00 and Kavya.

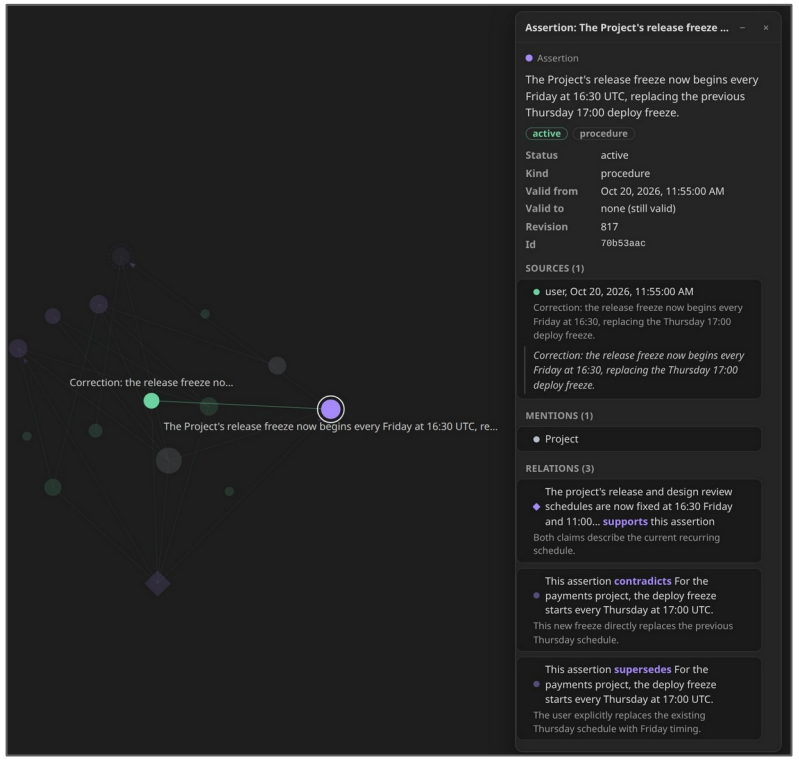


Figure 4. Correction: the graph shows Friday 16:30 as the current release freeze, linked to the old Thursday 17:00 value.

RESULTS AND OBSERVATIONS (1/3)

A corrected instruction carried into a new session.

In a three-session coding demo the required output changed from milliseconds to microseconds. In the next session the agent used microseconds without being told again.

Updating one assertion left the others alone.

A test conversation gave a person's city and food preference, then changed only the city. The system used the new city, kept the old one as history and still knew the food preference.

Every stored assertion can be traced to its message.

A database audit found that all 142 stored source quotes match their original text exactly.

RESULTS AND OBSERVATIONS (2/3)

Related assertions were combined into inferences.
The same audit found 8 inferences, each linked to at least two supporting assertions.

Dates are still a weak point.
Some relative dates were read wrongly. "Tomorrow" in a message sent on June 1 was stored as June 3.

The first model was too small.
Qwen3-1.7B did well on synthetic examples, but only 18 of 33 natural-conversation windows passed validation. Most failures were wrong source attribution, mismatched quotes and invalid IDs, so we widened the data, tightened the output format and moved to Qwen3-4B.

RESULTS AND OBSERVATIONS (3/3)

Qwen3-4B before and after fine-tuning, on 64 unseen tasks across 16 domains and 16 external samples

Measurement	Before	After
Correctly combined assertions (inferences)	1/16	13/16
Correct answers from supplied memories	10/16	15/16
Expected assertions recovered	50/55	41/55
Extracted assertions supported by the source	92.1%	90.5%
Corrections handled correctly	8/8	8/8
Outputs passing format checks	64/64	63/64
Expected assertions recovered (16 external samples)	16/40	13/40

Combining and answering improved a lot, but extraction recovered fewer assertions. A correct format also does not guarantee a correct meaning.

STATUS AND PENDING WORK

Module	What works now	Pending	Status
M1 Source Acquisition	Captures Pi messages and tool results; handles duplicates, offline queueing and retries. Covered by Pi integration tests.	API access control; tested backup and restore.	Implemented
M2 Knowledge Extraction	Extracts assertions with quotes, reviews corrections, builds embeddings and inferences. Extraction and review tests.	Better coverage, dates and source attribution; measure the review step.	Implemented; quality work pending
M3 Temporal Graph	Entity resolution, corrections, conflicts, history, stale inferences and the Neo4j copy. Integration tests for each.	Ambiguous updates, aliases and long correction chains.	Implemented; wider testing pending
M4 Hybrid Retrieval	Keyword, vector, time and graph search; rank fusion, packing, recall in Pi and cited answers.	Full LongMemEval-S run, ablations, quality and latency report.	Implemented; benchmark pending
M5 Model Training	Qwen3-4B fine-tuned with QLoRA, exported to Q8 GGUF and served by Ollama. Tested on 64 internal and 16 external cases.	Fix the extraction drop and date errors; more external tests and human review.	Trained; evaluation ongoing

REFERENCES (1/2)

[1] N. F. Liu et al., "Lost in the middle: How language models use long contexts," *Trans. Assoc. Comput. Linguist.*, vol. 12, pp. 157–173, 2024.

[2] D. Wu, H. Wang, W. Yu, Y. Zhang, K.-W. Chang, and D. Yu, "LongMemEval: Benchmarking chat assistants on long-term interactive memory," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2025, pp. 86809–86836.

[3] C. Packer et al., "MemGPT: Towards LLMs as operating systems," 2023, *arXiv:2310.08560*.

[4] P. Rasmussen, P. Paliychuk, T. Beauvais, J. Ryan, and D. Chalef, "Zep: A temporal knowledge graph architecture for agent memory," 2025, *arXiv:2501.13956*.

[5] P. Chhikara, D. Khant, S. Aryan, T. Singh, and D. Yadav, "Mem0: Building production-ready AI agents with scalable long-term memory," in *Proc. 28th Eur. Conf. Artif. Intell. (ECAI)*, 2025, pp. 2993–3000, doi: 10.3233/FAIA251160.

[6] S. Daga, S. Sreedhar, and D. Shah, "LongMemEval," Supermemory Research, May 2026. Accessed: Oct. 6, 2026. [Online]. Available: https://supermemory.ai/research/longmembench/

[7] C. Latimer et al., "Hindsight is 20/20: Building agent memory that retains, recalls, and reflects," 2025, *arXiv:2512.12818*.

[8] PostgreSQL Global Development Group, "Full text search," in *PostgreSQL 17 Documentation*, ch. 12, 2026. Accessed: Oct. 6, 2026. [Online]. Available: https://www.postgresql.org/docs/17/textsearch.html

REFERENCES (2/2)

[9] A. Kane et al., "pgvector: Open-source vector similarity search for Postgres," GitHub repository, 2026. Accessed: Oct. 6, 2026. [Online]. Available: https://github.com/pgvector/pgvector

[10] G. V. Cormack, C. L. A. Clarke, and S. Büttcher, "Reciprocal rank fusion outperforms Condorcet and individual rank learning methods," in *Proc. 32nd Int. ACM SIGIR Conf. Res. Develop. Inf. Retrieval*, Boston, MA, USA, 2009, pp. 758–759, doi: 10.1145/1571941.1572114.

[11] Neo4j, Inc., "Query the database," in *Neo4j HTTP API Manual*, v5, 2026. Accessed: Oct. 6, 2026. [Online]. Available: https://neo4j.com/docs/http-api/5/query/

[12] Y. Zhang et al., "Qwen3 Embedding: Advancing text embedding and reranking through foundation models," 2025, *arXiv:2506.05176*.

[13] M. Zechner et al., "pi: AI agent toolkit," GitHub repository, 2026. Accessed: Oct. 6, 2026. [Online]. Available: https://github.com/earendil-works/pi

[14] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "QLoRA: Efficient finetuning of quantized LLMs," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 36, 2023, pp. 10088–10115.

[15] A. Maharana, D.-H. Lee, S. Tulyakov, M. Bansal, F. Barbieri, and Y. Fang, "Evaluating very long-term conversational memory of LLM agents," in *Proc. 62nd Annu. Meeting Assoc. Comput. Linguist.*, Bangkok, Thailand, 2024, pp. 13851–13870, doi: 10.18653/v1/2024.acl-long.747.